End-to-End Guide: Connecting React Frontend to Employee Management System API

This guide provides a comprehensive, step-by-step procedure for connecting a modern React frontend (built with Vite) to your Node.js, Express, and MongoDB Employee Management backend.

1. System Architecture & Endpoints Overview

The frontend communicates with your Express server running on port 5000 through standard HTTP REST endpoints:

Action	HTTP Method	Endpoint URL	Payload / Params
Fetch All Employees	GET	http://localhost:5000/employee/allemployees	None
Add New Employee	POST	http://localhost:5000/employee/add-emp	{ name, email, phone, city, salary, bonus, deductions }
Calculate Net Salary	GET	http://localhost:5000/employee/salary/:id	:id (Employee ID param)
Update Employee	PUT	http://localhost:5000/employee/update/:id	:id param + updated JSON fields

Action	HTTP Method	Endpoint URL	Payload / Params
Delete Employee	DELETE	http://localhost:5000/employee/delete/:id	:id (Employee ID param)

2. Backend Configuration (CORS Setup)

Because the React development server runs on a different port (such as http://localhost:5173) than Express (http://localhost:5000), you must configure Cross-Origin Resource Sharing (CORS) in the backend to prevent the browser from blocking requests.

1. In your backend directory, install cors:

Bash

```
npm install cors
```

2. Open your backend server.js and apply the middleware **before** mounting your employee routes:

JavaScript

```
const express = require('express');

const dotEnv = require('dotenv');

const mongoose = require('mongoose');

const bodyParser = require('body-parser');

const cors = require('cors'); // 1. Import cors

const employeeRoutes = require('./routes/employeeRoutes');


dotEnv.config();

const app = express();

const PORT = process.env.PORT || 5000;
```

```
// Middleware
app.use(cors()); // 2. Enable CORS for cross-origin requests
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));

// Connect MongoDB & mount routes
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected successfully"))
  .catch((err) => console.error("MongoDB connection error:", err));

app.use('/employee', employeeRoutes);

app.listen(PORT, () => {
  console.log(`Server started and running at http://localhost:${PORT}`);
});
```

3. Frontend Project Setup

1. Initialize a new Vite-powered React project in a separate directory:

Bash

```
npm create vite@latest View -- --template react
cd View
```

2. Install axios for making HTTP network calls:

Bash

```
npm install axios
```

3. Set up the target directory structure inside View/src:

Plaintext

```
src/
 ├── components/
 │   ├── Navbar.jsx
 │   ├── Navbar.css
 │   ├── AddEmployee.jsx
 │   ├── AddEmployee.css
 │   ├── EmployeeList.jsx
 │   └── EmployeeList.css
 ├── App.jsx
 ├── App.css
 ├── main.jsx
 └── index.css
```

4. Frontend Component Implementation

4.1 Navigation Bar Component

src/components/Navbar.css

CSS

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #0f172a;
  padding: 16px 24px;
  border-radius: 8px;
  margin-bottom: 24px;
}
```

```
.navbar-brand {
  color: #f8fafc;
  font-size: 20px;
  font-weight: 700;
  margin: 0;
}

.navbar-links {
  display: flex;
  gap: 12px;
}

.nav-btn {
  background: transparent;
  color: #94a3b8;
  border: 1px solid #334155;
  padding: 8px 16px;
  border-radius: 6px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s ease-in-out;
}

.nav-btn:hover {
```

```
  background-color: #1e293b;
  color: #ffffff;
}

.nav-btn.active {
  background-color: #2563eb;
  color: #ffffff;
  border-color: #2563eb;
}
```

src/components/Navbar.jsx

JavaScript

```
import React from 'react';
import './Navbar.css';

function Navbar({ activeTab, setActiveTab }) {
  return (
    <nav className="navbar">
      <h2 className="navbar-brand">Employee Management System</h2>
      <div className="navbar-links">
        <button
          className={` nav-btn ${activeTab === 'list' ? 'active' : ''}` }
          onClick={() => setActiveTab('list')}
        >
          Employee List
        </button>
        <button
```

```
        className={` nav-btn ${activeTab === 'add' ? 'active' : ''}` }
        onClick={() => setActiveTab('add')}
       >
        Add Employee
       </button>
     </div>
    </nav>
  );
}

export default Navbar;
```

4.2 Add Employee Component (Insert / POST)

src/components/AddEmployee.css

CSS

```
.add-employee-card {
  max-width: 540px;
  margin: 0 auto;
  padding: 24px;
  background-color: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.07);
}

.card-title {
  margin: 0 0 16px 0;
```

```
  font-size: 18px;
  color: #1e293b;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px 16px;
}

.form-label {
  display: flex;
  flex-direction: column;
  font-size: 13px;
  font-weight: 600;
  color: #475569;
}

.form-label.full-width {
  grid-column: span 2;
}

.form-input {
  display: block;
  width: 100%;
  padding: 8px 10px;
```

```
  margin-top: 4px;
  font-size: 13px;
  border: 1px solid #cbd5e1;
  border-radius: 4px;
  box-sizing: border-box;
}

.form-input:focus {
  outline: none;
  border-color: #2563eb;
}

.submit-btn {
  width: 100%;
  margin-top: 18px;
  padding: 10px;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
}

.submit-btn:hover {
```

```
  background-color: #1d4ed8;
}
```

src/components/AddEmployee.jsx

JavaScript

```
import React, { useState } from 'react';
import axios from 'axios';
import './AddEmployee.css';

function AddEmployee({ onEmployeeAdded }) {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: '',
    city: '',
    salary: '',
    bonus: '',
    deductions: ''
  });

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
```

```
    await axios.post('http://localhost:5000/employee/add-emp', formData);
    alert('Employee Added Successfully!');
    setFormData({
      name: '',
      email: '',
      phone: '',
      city: '',
      salary: '',
      bonus: '',
      deductions: ''
    });
    if (onEmployeeAdded) onEmployeeAdded();
   } catch (error) {
    console.error('Error inserting data:', error);
    const serverMessage = error.response?.data?.error || error.response?.data?.message ||
error.message;
    alert(`Failed to add employee: ${serverMessage}`);
   }
  };

  return (
   <div className="add-employee-card">
    <h3 className="card-title">Add New Employee</h3>
    <form onSubmit={handleSubmit}>
     <div className="form-grid">
      <label className="form-label">
```

```
 Name *
 <input
  className="form-input"
  type="text"
  name="name"
  placeholder="Full name"
  value={formData.name}
  onChange={handleChange}
  required
 />
</label>

<label className="form-label">
 Email *
 <input
  className="form-input"
  type="email"
  name="email"
  placeholder="Email address"
  value={formData.email}
  onChange={handleChange}
  required
 />
</label>

<label className="form-label">
```

```
 Phone
 <input
  className="form-input"
  type="text"
  name="phone"
  placeholder="Phone number"
  value={formData.phone}
  onChange={handleChange}
 />
</label>

<label className="form-label">
 City
 <input
  className="form-input"
  type="text"
  name="city"
  placeholder="City"
  value={formData.city}
  onChange={handleChange}
 />
</label>

<label className="form-label">
 Salary (₹)
 <input
```

```
    className="form-input"
    type="number"
    name="salary"
    placeholder="Basic salary"
    value={formData.salary}
    onChange={handleChange}
   />
  </label>

  <label className="form-label">
   Bonus (₹)
   <input
    className="form-input"
    type="number"
    name="bonus"
    placeholder="Bonus"
    value={formData.bonus}
    onChange={handleChange}
   />
  </label>

  <label className="form-label full-width">
   Deductions (₹)
   <input
    className="form-input"
    type="number"
```

```
            name="deductions"
            placeholder="Deductions"
            value={formData.deductions}
            onChange={handleChange}
          />
        </label>
      </div>

      <button type="submit" className="submit-btn">
        Submit
      </button>
    </form>
  </div>
 );
}

export default AddEmployee;
```

4.3 Employee List Component (Retrieve, Calculate, Update, Delete)

src/components/EmployeeList.css

CSS

```
.list-container {
  width: 100%;
}

.list-title {
  font-size: 18px;
```

```
  color: #1e293b;
  margin-bottom: 14px;
}

.table-wrapper {
  overflow-x: auto;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
}

.employee-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
  font-size: 14px;
}

.employee-table th {
  background-color: #f8fafc;
  color: #475569;
  padding: 12px 14px;
  border-bottom: 2px solid #e2e8f0;
  font-weight: 600;
}

.employee-table td {
```

```
  padding: 10px 14px;
  border-bottom: 1px solid #f1f5f9;
}

.actions-cell {
  text-align: center;
  white-space: nowrap;
}

.btn-salary {
  padding: 5px 9px;
  margin-right: 6px;
  background-color: #10b981;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.btn-edit {
  padding: 5px 9px;
  margin-right: 6px;
  background-color: #f59e0b;
  color: #ffffff;
  border: none;
  border-radius: 4px;
```

```
  cursor: pointer;
}

.btn-delete {
  padding: 5px 9px;
  background-color: #ef4444;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

/* Edit Modal */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100vw;
  height: 100vh;
  background-color: rgba(0, 0, 0, 0.45);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}
```

```
.modal-content {
  background-color: #ffffff;
  padding: 24px;
  border-radius: 8px;
  width: 360px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}

.modal-form {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.modal-input {
  width: 100%;
  padding: 6px 8px;
  margin-top: 3px;
  box-sizing: border-box;
  border: 1px solid #cbd5e1;
  border-radius: 4px;
}

.modal-actions {
  display: flex;
  justify-content: flex-end;
```

```
  gap: 8px;
  margin-top: 10px;
}

.btn-cancel {
  padding: 6px 12px;
  background-color: #94a3b8;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.btn-save {
  padding: 6px 12px;
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

src/components/EmployeeList.jsx

JavaScript

```
import React, { useState } from 'react';
import axios from 'axios';
import './EmployeeList.css';
```

```
function EmployeeList({ employees, loading, onRefresh }) {
  const [editingEmployee, setEditingEmployee] = useState(null);

  // 1. GET Net Salary
  const handleCalculateSalary = async (employeeId) => {
    try {
      const response = await
axios.get(`http://localhost:5000/employee/salary/${employeeId}`);
      const data = response.data;

      const empName = data.name || data.employee?.name || 'Employee';
      const baseSalary = data.baseSalary ?? data.salary ?? 0;
      const bonus = data.bonus ?? 0;
      const deductions = data.deductions ?? 0;
      const netSalary = data.netSalary ?? 0;

      alert(
        `Employee: ${empName}\n` +
        `Base Salary: ₹${baseSalary}\n` +
        `Bonus: ₹${bonus}\n` +
        `Deductions: ₹${deductions}\n` +
        `-------------------------\n` +
        `Net Salary: ₹${netSalary}`
      );
    } catch (error) {
```

```
    console.error('Error fetching net salary:', error);
    alert('Could not calculate net salary.');
  }
};

// 2. DELETE Employee
const handleDelete = async (id) => {
  const confirmDelete = window.confirm('Are you sure you want to delete this employee?');
  if (!confirmDelete) return;

  try {
    await axios.delete(`http://localhost:5000/employee/delete/${id}`);
    alert('Employee deleted successfully!');
    if (typeof onRefresh === 'function') {
      onRefresh();
    }
  } catch (error) {
    console.error('Error deleting employee:', error);
    const errMsg = error.response?.data?.message || error.message;
    alert(`Failed to delete employee: ${errMsg}`);
  }
};

// 3. EDIT Actions (PUT)
const handleEditClick = (emp) => {
  setEditingEmployee({ ...emp });
```

```
};

const handleEditChange = (e) => {
 setEditingEmployee({
  ...editingEmployee,
  [e.target.name]: e.target.value
 });
};

const handleUpdateSubmit = async (e) => {
 e.preventDefault();
 try {
  await axios.put(
   `http://localhost:5000/employee/update/${editingEmployee._id}`,
   editingEmployee
  );
  alert('Employee updated successfully!');
  setEditingEmployee(null);
  if (typeof onRefresh === 'function') {
   onRefresh();
  }
 } catch (error) {
  console.error('Error updating employee:', error);
  alert('Failed to update employee.');
 }
};
```

```
if (loading) return <p>Loading employee data...</p>;

return (
 <div className="list-container">
  <h3 className="list-title">Employee Directory</h3>

  {Array.isArray(employees) && employees.length === 0 ? (
   <p>No employees found in database.</p>
  ) : (
   <div className="table-wrapper">
    <table className="employee-table">
     <thead>
      <tr>
       <th>Name</th>
       <th>Email</th>
       <th>City</th>
       <th>Salary (₹)</th>
       <th>Bonus (₹)</th>
       <th>Deductions (₹)</th>
       <th style={{ textAlign: 'center' }}>Actions</th>
      </tr>
     </thead>
     <tbody>
      {Array.isArray(employees) &&
       employees.map((emp) => (
```

```
<tr key={emp._id}>
 <td>{emp.name}</td>
 <td>{emp.email}</td>
 <td>{emp.city || 'N/A'}</td>
 <td>{emp.salary}</td>
 <td>{emp.bonus}</td>
 <td>{emp.deductions}</td>
 <td className="actions-cell">
  <button
   className="btn-salary"
   onClick={() => handleCalculateSalary(emp._id)}
  >
   Salary
  </button>
  <button
   className="btn-edit"
   onClick={() => handleEditClick(emp)}
  >
   Edit
  </button>
  <button
   className="btn-delete"
   onClick={() => handleDelete(emp._id)}
  >
   Delete
  </button>
```

```
          </td>
         </tr>
        ))}
     </tbody>
   </table>
  </div>
)}

{/* Edit Modal Overlay */}
{editingEmployee && (
  <div className="modal-overlay">
    <div className="modal-content">
      <h3 style={{ marginTop: 0 }}>Edit Employee</h3>
      <form onSubmit={handleUpdateSubmit} className="modal-form">
        <label>
          Name:
          <input
            className="modal-input"
            type="text"
            name="name"
            value={editingEmployee.name}
            onChange={handleEditChange}
            required
          />
        </label>
```

```
<label>
 Email:
 <input
  className="modal-input"
  type="email"
  name="email"
  value={editingEmployee.email}
  onChange={handleEditChange}
  required
 />
</label>

<label>
 City:
 <input
  className="modal-input"
  type="text"
  name="city"
  value={editingEmployee.city || ''}
  onChange={handleEditChange}
 />
</label>

<label>
 Salary (₹):
 <input
```

```
    className="modal-input"
    type="number"
    name="salary"
    value={editingEmployee.salary || ''}
    onChange={handleEditChange}
   />
  </label>

  <label>
   Bonus (₹):
   <input
    className="modal-input"
    type="number"
    name="bonus"
    value={editingEmployee.bonus || ''}
    onChange={handleEditChange}
   />
  </label>

  <label>
   Deductions (₹):
   <input
    className="modal-input"
    type="number"
    name="deductions"
    value={editingEmployee.deductions || ''}
```

```
            onChange={handleEditChange}
           />
          </label>

          <div className="modal-actions">
           <button
            type="button"
            className="btn-cancel"
            onClick={() => setEditingEmployee(null)}
           >
            Cancel
           </button>
           <button type="submit" className="btn-save">
            Save Changes
           </button>
          </div>
         </form>
        </div>
       </div>
     )}
   </div>
  );
}

export default EmployeeList;
```

4.4 Main Entry Component

src/App.css

CSS

```
.app-container {
  max-width: 960px;
  margin: 0 auto;
  padding: 24px;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
  color: #1e293b;
}

.main-content {
  margin-top: 8px;
}
```

src/App.jsx

JavaScript

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import Navbar from './components/Navbar';
import AddEmployee from './components/AddEmployee';
import EmployeeList from './components/EmployeeList';
import './App.css';

function App() {
  const [activeTab, setActiveTab] = useState('list');
```

```
const [employees, setEmployees] = useState([]);
const [loading, setLoading] = useState(false);

// Fetch all employees from Node API
const fetchEmployees = async () => {
 setLoading(true);
 try {
  const response = await axios.get('http://localhost:5000/employee/allemployees');
  if (Array.isArray(response.data)) {
   setEmployees(response.data);
  } else {
   setEmployees([]);
  }
 } catch (error) {
  console.error('Error fetching employees:', error);
  setEmployees([]);
 } finally {
  setLoading(false);
 }
};

useEffect(() => {
 fetchEmployees();
}, []);

const handleEmployeeAdded = () => {
```

```
    fetchEmployees();
    setActiveTab('list');
  };

  return (
    <div className="app-container">
      <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

      <main className="main-content">
        {activeTab === 'list' && (
          <EmployeeList
            employees={employees}
            loading={loading}
            onRefresh={fetchEmployees}
          />
        )}

        {activeTab === 'add' && (
          <AddEmployee onEmployeeAdded={handleEmployeeAdded} />
        )}
      </main>
    </div>
  );
}

export default App;
```

5. Execution & Verification

Run both applications concurrently in separate terminals:

1. **Terminal 1 (Backend):**

Bash

cd NodeMongo

npm start

Expected output:

Plaintext

Server started and running at http://localhost:5000

MongoDB connected successfully

2. **Terminal 2 (Frontend):**

Bash

cd View

npm run dev

Expected output:

Plaintext

VITE ready in 250 ms

➜ Local: http://localhost:5173/

3. **In the Browser:**
    - Open http://localhost:5173/.
    - Click **Add Employee** → submit test employee details. You will be alerted with "Employee Added Successfully!" and redirected to the list.
    - On **Employee List**, test clicking **Salary**, **Edit**, and **Delete** to verify all CRUD cycles against MongoDB.